# System Calls for File Management

**Definition**

**File-management system calls** are system calls used to create, open, read, write, reposition, inspect, and close files. They provide the basic interface between programs and the file system.

## 1. Basic Idea

**Main Concept**

Many system calls are related to the file system. Some operate on **individual files**, while others work on **directories** or the **file system as a whole**.

**This Section**

Here the focus is on system calls that operate on **individual files**.

## 2. Opening a File

**Open**

Before a file can be read or written, it must first be **opened**. The `open` call specifies:

- the file name,
- the access mode,
- and optionally file-creation behavior.

**File Name**

A file may be opened using:

- an **absolute path name**, or
- a path relative to the **working directory**.

**Access Modes**

Common access modes are:

- `O_RDONLY` — open for reading only
- `O_WRONLY` — open for writing only
- `O_RDWR` — open for both reading and writing

**Creating a File**

To create a new file while opening, the parameter `O_CREAT` is used.

## 3. File Descriptor

**Definition**

When a file is opened successfully, the operating system returns a small integer called a **file descriptor**.

**Use of File Descriptor**

This file descriptor is then used in later operations such as:

- reading,
- writing,
- repositioning,
- closing the file.

## 4. Closing a File

**Close**

After file operations are finished, the file should be closed using `close()`.

**Why Close is Important**

Closing a file releases the file descriptor so that it may be reused by a later `open()` call.

## 5. Reading and Writing Files

**Most Common Calls**

The most heavily used file-management system calls are:

- `read`
- `write`

**read Example**

```
count = read(fd, buffer, nbytes);
```

**write Example**

```
count = write(fd, buffer, nbytes);
```

**Idea**

Both `read` and `write` use similar parameters:

- file descriptor,
- buffer address,
- number of bytes.

## 6. Sequential and Random Access

**Sequential Access**

Most programs read and write files **sequentially**. This means data are processed in order, one part after another.

**Random Access**

Some programs need **random access**, meaning they must jump directly to any location inside the file instead of moving only in sequence.

## 7. File Position Pointer

**Position Pointer**

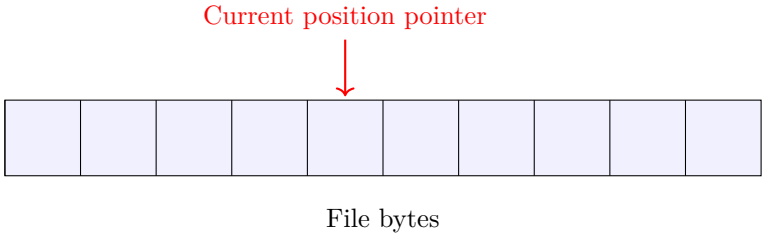
Each open file has an associated **position pointer**. It indicates the current location in the file.

**Meaning**

During sequential reading or writing, the position pointer usually points to:

- the next byte to be read, or
- the next byte to be written.

### 7.1 Simple Pointer Diagram



## 8. `lseek()` System Call

### Meaning of `lseek`

The `lseek()` call changes the value of the file position pointer. It allows later `read()` or `write()` calls to start at a different place in the file.

### General Form

```
pos = lseek(fd, offset, whence);
```

### Parameters of `lseek`

- first parameter — file descriptor
- second parameter — desired file position / offset
- third parameter — reference point for the offset

### Reference Point

The third parameter tells whether the offset is measured relative to:

- the **beginning** of the file,
- the **current position**,
- or the **end** of the file.

### Return Value

The value returned by `lseek()` is the **absolute position** in the file, measured in bytes, after the pointer has been changed.

## 9. File Information

### Stored Information

For every file, UNIX stores information such as:

- file mode,
- file size,
- time of last modification,
- and other related details.

**File Mode**

The **file mode** tells what kind of file it is, such as:

- regular file,
- special file,
- directory.

## 10. `stat()` and `fstat()`

**Meaning**

Programs can inspect file information using:

- `stat()`
- `fstat()`

**`stat()`**

`stat()` takes:

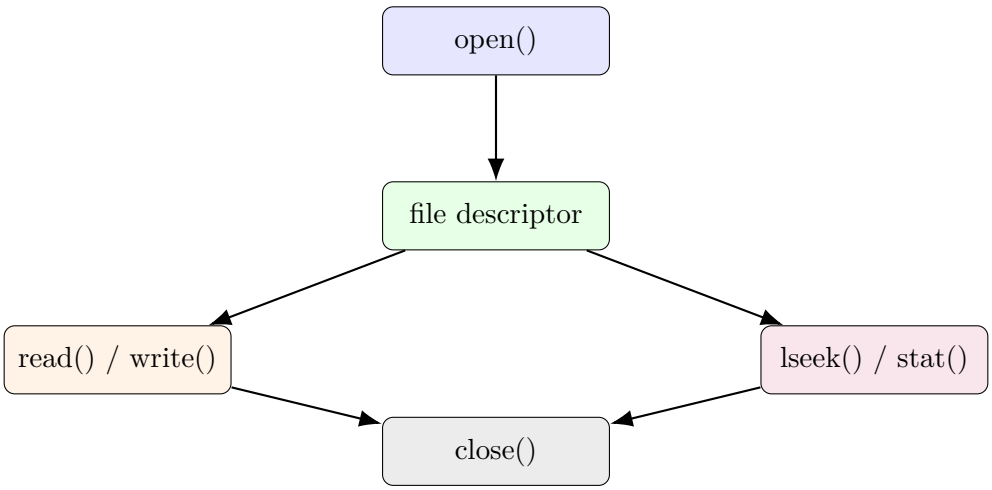
- a file name,
- and a pointer to a structure where the information will be stored.

**`fstat()`**

`fstat()` works in a similar way, but it is used for an **already open file**.

## 11. Flow of File Operations

### 11.1 Simple Flow Diagram



## 12. Main Calls at a Glance

| Call | Purpose |
|---|---|
| `open()` | Open a file and obtain a file descriptor |
| `close()` | Close a file descriptor |
| `read()` | Read bytes from a file |
| `write()` | Write bytes to a file |
| `lseek()` | Change the file position pointer |
| `stat()` | Get information about a file by name |
| `fstat()` | Get information about an open file |

## 13. Conclusion

**Summary**

System calls for file management allow programs to open files, read and write data, move to any position inside a file, inspect file information, and close files afterward. These calls form the basic operating-system support for working with files.